

CADCrafter: Generating Computer-Aided Design Models from Unconstrained Images

Cheng Chen^{1,2*}, Jiacheng Wei^{1*}, Tianrun Chen^{3,7†}, Chi Zhang⁵, Xiaofeng Yang¹, Shangzhan Zhang⁷, Bingchen Yang¹, Chuan-Sheng Foo^{2,6}, Guosheng Lin¹, Qixing Huang⁴, Fayao Liu^{2†}

¹Nanyang Technological University, ²Institute for Infocomm Research, A*STAR, Singapore

³KOKONI3D, Moxin (Huzhou) Technology Co., LTD., ⁴The University of Texas at Austin

⁵Weatlake University, ⁶Centre for Frontier AI Research, A*STAR, Singapore

⁷Zhejiang University

{cheng021, jiacheng.wei}@ntu.edu.sg, tianrun.chen@kokoni3d.com, fayao.liu@gmail.com

Abstract

Creating CAD digital twins from the physical world is crucial for manufacturing, design, and simulation. However, current methods typically rely on costly 3D scanning with labor-intensive post-processing. To provide a user-friendly design process, we explore the problem of reverse engineering from unconstrained real-world CAD images that can be easily captured by users of all experiences. However, the scarcity of real-world CAD data poses challenges in directly training such models. To tackle these challenges, we propose CADCrafter, an image-to-parametric CAD model generation framework that trains solely on synthetic textureless CAD data while testing on real-world images. To bridge the significant representation disparity between images and parametric CAD models, we introduce a geometry encoder to accurately capture diverse geometric features. Moreover, the texture-invariant properties of the geometric features can also facilitate the generalization to real-world scenarios. Since compiling CAD parameter sequences into explicit CAD models is a non-differentiable process, the network training inherently lacks explicit geometric supervision. To impose geometric validity constraints, we employ direct preference optimization (DPO) to fine-tune our model with the automatic code checker feedback on CAD sequence quality. Furthermore, we collected a real-world dataset, comprised of multi-view images and corresponding CAD command sequence pairs, to evaluate our method. Experimental results demonstrate that our approach can robustly handle real unconstrained CAD images, and even generalize to unseen general objects.

1. Introduction

Computer-aided design (CAD) provides fundamental mechanical components that are essential to create shapes and mechanisms in all manufacturing and design applications. Parametric CAD command sequences enable precise control over shapes and facilitate effortless future modifications on size and scales. However, manual creation of CAD command sequences is tedious and time-consuming, leading to reverse engineering studies to recover CAD design procedures from existing CAD models. Current research focuses predominantly on reconstructing CAD command sequences from 3D representations such as B-Reps [34, 37], point clouds [20, 36], and voxels [14, 15]. These forms are typically derived from synthetic digital data or from high-quality 3D reconstructions obtained using costly 3D sensors. This dependency on sophisticated data and expensive technology limits the feasibility of these methods in practical everyday applications.

Recent advances in generative models have facilitated the generation and reconstruction of images to 3D [10, 16–18] with ease. However, these methods often yield 3D shapes with rough surfaces and indistinct, blurred edges, failing to accurately replicate geometric standards such as rectangles or circles. Moreover, the 3D shapes produced by these methods are difficult to edit and lack the precision required for direct use in manufacturing.

This prompts us to investigate the feasibility of directly generating editable CAD command sequences from images, as illustrated in Figure 1. However, the task is particularly difficult due to the significant representational and domain gap between the two modalities, where CAD commands consist of a mix of discrete geometric operations and continuous parameters while images capture raw appearance with limited spatial information. The challenge is amplified with in-the-wild, unconstrained images. These images

* The first two authors contributed equally to this work.

† Corresponding authors.

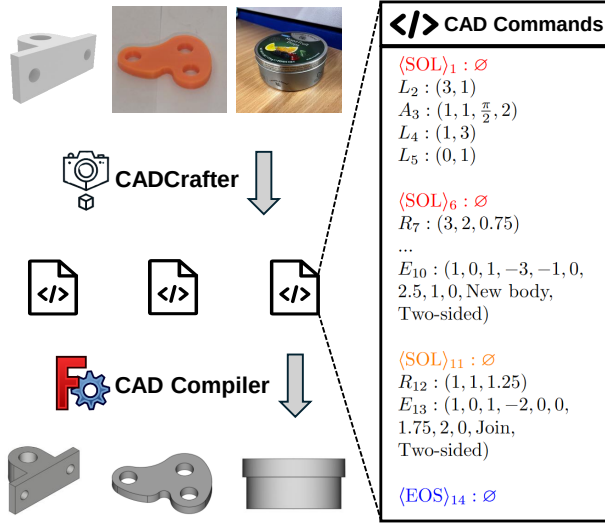


Figure 1. Our proposed CADCrafter can generate CAD command sequences from unconstrained multi-domain images, including (from left to right) synthetic data renderings, 3D-printed CAD models, and unseen general objects. These generated CAD commands can then be compiled into 3D CAD models. Notably, our model is trained solely on synthetic data renderings.

frequently exhibit variability in camera poses, lighting conditions, and noise, as well as various materials and textures of the objects depicted. Another challenge in developing this system lies in the difficulty of collecting paired CAD commands and image data from real-world scenarios, making it necessary to rely on synthetic datasets for training. However, models trained solely on synthetic data often underperform with real-world data. Therefore, there is a critical need for a method to bridge this gap, ensuring that approaches trained on synthetic data can perform effectively on both synthetic and real-world unconstrained images.

To tackle these challenges, we propose CADCrafter, a method engineered to directly generate CAD command sequences from both multi-view and single-view unconstrained images. Specifically, as shown in Figure 2, we first train a transformer-based autoencoder to map the CAD tokens to a latent space and then reconstruct them. Then, we apply a latent diffusion transformer to denoise the latent CAD codes conditioned on the input images. Unlike traditional latent diffusion architectures [42] that rely directly on image features as conditions, our approach utilizes geometric features of images, specifically depth and normal maps. There are two main benefits of geometric features: they enhance geometry representations to boost accurate command prediction and are invariant to the textural gap between synthetic data and real-world images. Additionally, different modalities of geometry features capture various perspectives of an object, with each modal providing unique geometric information. To capitalize on this, we designed a geometry encoder that adaptively consolidates geometric data

from each modality.

When generating CAD commands, the non-differentiable nature of the CAD compiler makes it inherently challenging to directly incorporate geometric constraints. Due to the geometry precision required in CAD models, inaccurate commands may fail to compile into a valid CAD model, as shown in Figure 3.

To implicitly learn the correct shape pattern of CAD models, we enhance the latent diffusion model with additional constraints and regularization, c.f. [9]. Drawing inspiration from reinforcement learning with human feedback (RLHF) [4], we implement a code checker to improve the validity of denoised latent codes. Specifically, we deploy the CAD compiler as an automatic checker to categorize codes as valid or invalid. Subsequently, we fine-tune the diffusion model using these categorized sets through direct preference optimization [23, 31] to improve the generation quality and accuracy.

Since single-view images inherently lack complete information about the unseen part of a 3D object, instead of training separate models for multi-view and single-view inputs, we distill the comprehensive knowledge from our pre-trained multi-view geometry encoder into a single-view geometric encoder by aligning their feature representations. This enables the model to learn the mapping from single-view to multi-view input, thereby enhancing both the accuracy and robustness when processing single-image inputs.

To validate our method, we collected RealCAD, a real-world dataset pairing CAD commands with multi-view images, captured freely on CAD models fabricated using 3D printing technology with various materials and textures.

Our contributions are: (a) We introduce CADCrafter, a latent diffusion-based framework to generate parametric CAD models from unconstrained images, leveraging geometric features to mitigate the domain gap between synthetic training data and in-the-wild testing data. (b) We introduce an automatic code checker to learn CAD geometry validity by fine-tuning our diffusion model with direct preference optimization (DPO) thereby improving accuracy and reducing invalid outputs. (c) Our proposed CADCrafter framework accommodates both single-view and multi-view inputs. In addition, we introduce a dataset of unconstrained 3D printed CAD images paired with CAD commands, demonstrating the robustness and generalizability of the model.

2. Related work

Generative models for CAD. Most existing CAD generation research focuses on unconditional generation [40] or conditional generation based on complete 3D information, such as point clouds [20, 36], sketches [13, 32], B-reps [34, 37] and voxel grids [14, 15]. For instance, DeepCAD [36] utilizes an autoencoder to encode CAD models and em-

plays GANs for unconditional generation. SkexGen [38] introduces an autoregressive generative model that encodes CAD construction sequences into disentangled codebooks. HNC-CAD [39] represents CAD models as a hierarchical tree of three levels of neural codes. Draw Step by Step [20] incorporates a tokenizer to compress CAD point clouds and trains a multi-modal diffusion model for point cloud-conditioned generation. CAD-SIGNet [11] proposes a layer-wise cross-attention mechanism between point clouds and CAD sequence embeddings. MultiCAD [19] develops a multimodal contrastive learning strategy to align CAD sequences with point clouds. More recently, Text2CAD [12] has been introduced to generate parametric CAD models from text instructions. Img2CAD [7] is able to generate CAD command sequences with image inputs, however, it adopts a discriminative framework which results in limited performance [5], especially on real-world objects.

Current CAD construction sequence data sets like DeepCAD [36] and Fusion360 [35] are standard synthetic data sets. These models are typically trained and tested on noise-free synthetic data. In this paper, we present a dataset that pairs multi-view images with CAD sequences and explores a more user-friendly approach by training only on synthetic data and evaluating our model on both synthetic and real-world captured data.

3D Generative Models. With the rise of large-scale model training, recent advances in 3D generative models have been significant. Most existing methods generate 3D shapes in discrete forms, such as implicit neural fields [6, 8, 10, 22], point clouds [41], and meshes [17, 26, 27, 33]. However, these generated shapes often suffer from a lack of sharp geometric features and are not directly editable by users.

In contrast to these works, our model directly outputs sequences of CAD operations that can be readily imported into any CAD tool [1–3] for user editing.

3. Approach

In this chapter, we present CADCrafter, a latent diffusion-based transformer tailored for generating CAD command sequences from images. Trained on a synthetic dataset and evaluated on both synthetic and real-world data, the model incorporates a geometry conditioning encoder to enhance geometric understanding and generalization. Additionally, we have developed a multi-view to single-view distillation technique to improve robustness for single-view inputs and introduced an annotation-free direct preference optimization method to improve accuracy in CAD representations.

3.1. CAD Command Sequence Encoding

The comprehensive CAD toolkit features an extensive array of commands, yet only a limited subset is frequently utilized in practice. Drawing on previous research [35, 36], we focus on two commonly used categories: *sketch*

and *extrusion*, which offer ample expressive capabilities. We present an example of a simple CAD command sequence in Figure 1. For simplicity, in *sketch*, we adopt commands $\{\langle \text{SOL} \rangle, L, A, R\}$, namely *start*, *line*, *arc*, and *circle*, to draw curves forming enclosed 2D regions named *profile*. Then, each 2D *profile* can be lifted to a 3D body using the *extrusion* command *E*. Each of these discrete commands is defined by its unique continuous parameters, which determine the size, location, scale, and type.

Following the approach in DeepCAD [36], we normalize all CAD models and quantize the continuous parameters into 256 levels represented as 8-bit integers to process the discrete and continuous command sequences. The i -th line of command C_i is represented by the one-hot encoding of command types s_i and a stacking of all parameters for all command types into a vector p_i , setting the unused parameters to -1 . We then pad the sequence to a fixed length, N_c , using the empty command $\langle \text{EOS} \rangle$. To process CAD commands in a manner similar to natural language processing [30], we tokenize the commands by mapping them to embedding spaces, the resulting embedding $e(C_i) = e_i^{\text{cmd}} + e_i^{\text{param}} + e_i^{\text{pos}} \in \mathbb{R}^{d_E}$, where e_i^{pos} is a readable positional embedding and $d_E = 256$ is the embedding dimension. More details on the CAD commands and the tokenization process are provided in the supplementary.

To facilitate the generation of sequential CAD data conditioned on images, as depicted in Figure 2, we initially train a transformer-based autoencoder to encode CAD sequences into latent vectors z and subsequently reconstruct them. Then we adopt the latent diffusion framework [25], which allows efficient learning and sampling within the latent CAD space with image conditioning. The design and training details of the autoencoder are similar to DeepCAD [36] and are included in the supplementary materials.

3.2. Geometry Conditioning Encoder

CAD commands are precise operations based on geometry structures; thus, it is important to explore more geometric information from the input image. Therefore, we extract depth and surface normal maps with geometry estimation models [43]. Additionally, depth and normal are invariant to textures, which significantly reduces the domain gap between our texture-less synthetic renderings used for training and the unconstrained images used for testing.

Specifically, we feed the extracted depth and normal maps to the pre-trained DINO-V2[21] encoder to get DINO features $h_i^{\text{depth}}, h_i^{\text{normal}} \in \mathbb{R}^{d_{\text{dino}}}$ where $i \in \{0, 1, 2, 3\}$ indicates different views and $d_{\text{dino}} = 1536$.

We designed a transformer-based geometry encoder that adaptively consolidates geometric cues from each view and modality, as each provides unique geometric information about the object. To help the model more effectively learn to integrate information from different modalities, the DINO

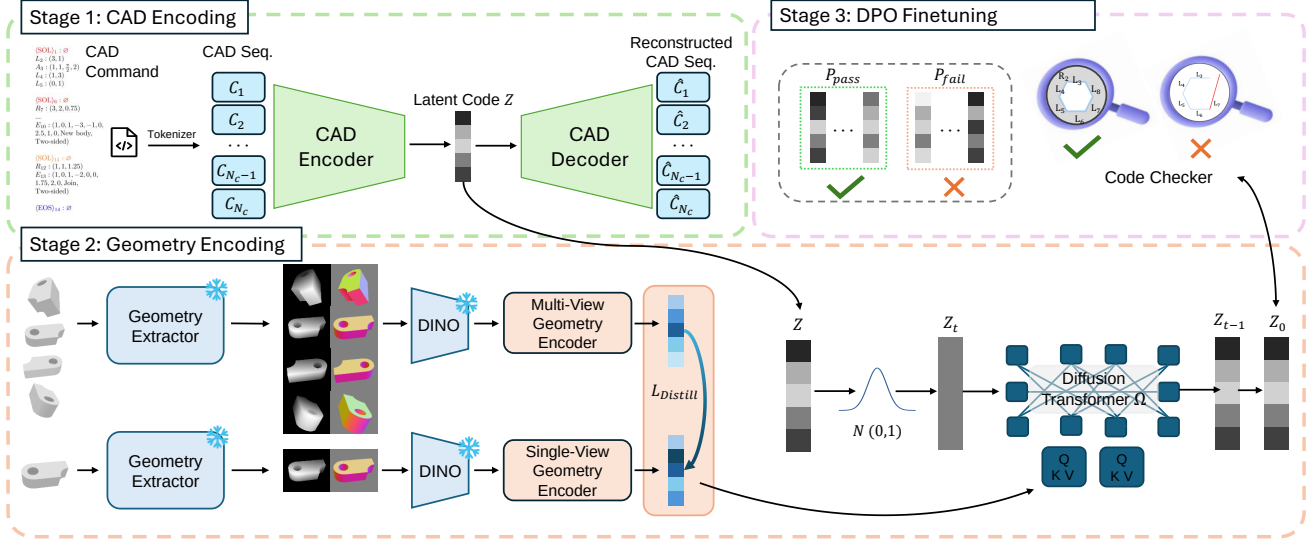


Figure 2. The training pipeline comprises three stages. In the first, a transformer autoencoder reconstructs CAD command sequences into a latent space. Second, we extract depth and normal using a pre-trained geometric extractor, the encoded features serve as conditions in the latent diffusion model; the multi-view geometric encoders and the latent diffusion model are jointly trained. Later, a single-view geometry encoder is trained by distilling knowledge from the multi-view encoder to enhance robustness. Third, we develop a geometry validity-based code checker and fine-tune the diffusion model with direct preference optimization (DPO) to improve generation quality and accuracy.

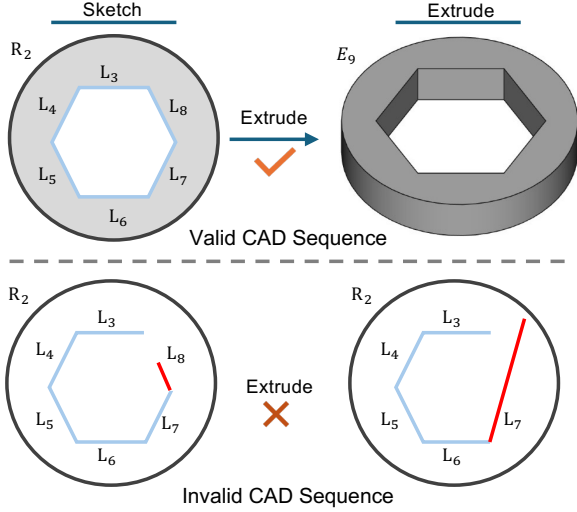


Figure 3. The code checker checks if the generated CAD command sequence is compilable. The first row illustrates cases that can be successfully compiled while the second row shows invalid cases where no 2D profile is enclosed by the curves. The compiler inherently performs as an automatic checker to help our DPO fine-tuning process.

features are stacked as patches and add a learnable modality embedding e to get:

$$\mathbf{H} = \text{cat}_{i=0}^3 (h_i^{\text{depth}} + e^{\text{depth}}, h_i^{\text{normal}} + e^{\text{normal}}), \quad (1)$$

where $\mathbf{H} \in \mathbb{R}^{8 \times d_{\text{dino}}}$ and cat denotes the concatenation operation across all views and modalities. We further apply a

rotary positional embedding [28] to each token to help the multi-view geometry encoder effectively combine information. The averaged output feature f_m of the geometry encoder is used as the conditioning vector.

3.3. Denoising CAD Latent Vectors

Since our sample space is the latent vectors z generated by the transformer-based autoencoder, unlike conventional DDPM architectures that employ UNet structures tailored for image processing, we implement a diffusion transformer architecture Ω to denoise the latent vector. The diffusion transformer architecture is similar to that of DALL-E 2 [24], which consists of layers with attention mechanisms [30], fully connected layers, and layer normalization.

Given the sampled CAD latent z , at each iteration, we add noise corresponding to a random timestep t to it to obtain z_t . The model then learns to restore the original latent z_0 . The diffusion model takes z_t , f_m and $\gamma(t)$ as inputs, where $\gamma(t)$ is a positional embedding of timestep t .

In our experiments, we found that directly predicting the original z_0 yields better performance than predicting the added noise. Therefore, the loss function is defined as:

$$\mathcal{L}_{\text{diff}} = \|\Omega(z_t, \gamma(t)|f_m) - z_0\|^2. \quad (2)$$

During testing, we begin with a randomly sampled noise vector $z_T \sim \mathcal{N}(0, I)$ and iteratively apply our diffusion model to denoise it, ultimately producing the final output z_0 . The generated latents z_0 are passed to the previously trained decoder to obtain the reconstructed CAD sequence.

3.4. Multi-View to Single-View Distillation

Creating a 3D object from a single-view input introduces inherent ambiguities, as the model must infer details from unseen areas. Instead of training a separate model for the single-view setting, we adopt the model trained from multi-view and distill the knowledge from the multi-view geometry encoder to the single-view geometry encoder by implicitly reducing the distance between the condition features.

Specifically, in single-view training, we freeze the weights of the trained multi-view geometry encoder as the reference model. We continue to feed multi-view inputs and employ a distillation loss to distill knowledge from it:

$$\mathcal{L}_{\text{distill}} = 1 - \frac{f_s \cdot f_m}{\|f_s\| \|f_m\|}, \quad (3)$$

where f_m is the averaged output feature of the multi-view geometry encoder and f_s is the averaged output feature of the single-view geometry encoder. The single-view geometry encoder is updated with $\mathcal{L}_{\text{distill}}$ and $\mathcal{L}_{\text{diff}}$.

3.5. Direct Preference Optimization based CAD Code Checker

During training, diffusion loss concentrates on aligning distributions without explicit geometric supervision. However, CAD compilers enforce strict command rules. As shown in Figure 3, there are instances where curves do not form closed surfaces, causing the generated CAD code to fail compiler checks.

To improve geometric precision, we take inspiration from reinforcement learning from human feedback (RLHF) [4] in large language models (LLMs), where a reward function is trained from comparison data on model output to represent human preferences, and reinforcement learning is used to align the policy model. Our key idea is to introduce a code checker to serve as an implicit reward model for the denoised latent code, utilizing direct preference optimization (DPO) [23, 31] to fine-tune our approach.

Specifically, we generate multiple latent z vectors and use a CAD compiler to verify the compilability of the decoded commands. This automatic process enables us to pick a set of valid latent vectors (positive set) and a set of invalid latent vectors (negative set). We then fine-tune the diffusion model using the DPO loss, which is defined as follows:

$$L(\theta) = -\mathbb{E}_{(z_0^w, z_0^l) \sim \mathcal{D}, t \sim \mathcal{U}(0, T), z_t^w \sim q(z_t^w | z_0^w), z_t^l \sim q(z_t^l | z_0^l)} \log \sigma \left(-\frac{\beta}{2} (\|\epsilon^w - \epsilon_\theta(z_t^w, t)\|_2^2 - \|\epsilon^w - \epsilon_{\text{ref}}(z_t^w, t)\|_2^2 \right. \\ \left. - (\|\epsilon^l - \epsilon_\theta(z_t^l, t)\|_2^2 - \|\epsilon^l - \epsilon_{\text{ref}}(z_t^l, t)\|_2^2) \right). \quad (4)$$

where ϵ is the noise in diffusion process, the loss term $l_w = \|\epsilon_w - \epsilon_\theta(z_t^w, t)\|$ represents the model preference toward the positive sample z_w , while $l_l = \|\epsilon_l - \epsilon_{\text{ref}}(z_t^l, t)\|$

represents the model’s preference toward the negative sample z_l . It can be observed that when optimizing the DPO loss, l_w decreases while l_l increases. This adjustment increases the probability of generating positive samples and decreases the probability of producing code that fails the checks.

Furthermore, $\epsilon_{\text{ref}}()$ denotes the frozen pretrained diffusion model trained in the second phase, and $\epsilon_\theta()$ is the updating network. By limiting the difference between the finetuned model’s output and the pre-trained model, the effective knowledge acquired during pretraining can be preserved. The parameter β controls the regularization of the fine-tuned model’s distance from the original model. A larger β imposes more constraints when the model deviates from the pre-trained model. In our experiments, we set $\beta = 20$.

4. Experiments

4.1. Experimental Setups

Datasets. We train our method solely on **DeepCAD** [36] training set, a dataset composed mainly of CAD mechanical parts. We render 8 sets of 4-view images around the CAD model with a random elevation and perturbations on azimuth. For the single-view setting, we randomly pick one image for each training step.

To better assess the generalizability of the model, we collect our own real-world testing dataset **RealCAD**. We randomly select 150 CAD models from the DeepCAD test set and fabricate them with 3D printing using various textures and materials. As shown in Figure 4, we casually take 4-view images around the printed objects without specific requirements.

Implementation Details. All experiments are performed on a single RTX6000 Ada GPU. In the first stage, we train the autoencoder for 1,000 epochs using a learning rate of 2×10^{-4} . In the second stage, we train the image-conditioned diffusion model with a batch size of 2,048 for 3,000 epochs using a learning rate of 5×10^{-5} . In the DPO finetuning stage, we collect 10000 pairs of positive and neg-

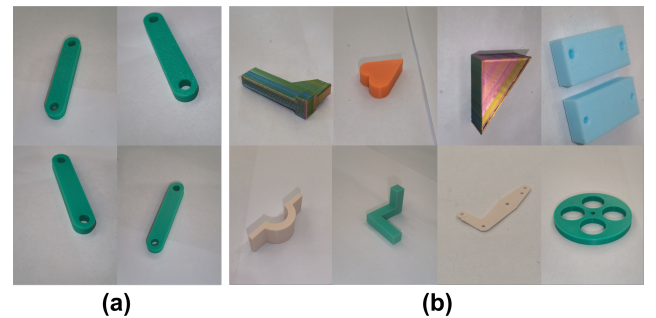


Figure 4. We showcase our RealCAD dataset: (a) casually captured multi-view images of a 3D printed CAD model, (b) more examples of 3D printed CAD models freely captured with iPhones.

ative pairs and use them to further train the diffusion model for 500 epochs. We uniformly use the pre-trained Metric3D [43] as depth and normal extractors.

Evaluation Metrics. Following previous work [20, 36], we adopt Command Accuracy (Acc_{cmd} in %), which measures the correctness of the predicted CAD command types, and Parameter Accuracy (Acc_{para} in %), which measures the correctness of the command parameters once the command type is correctly recovered. Both Acc_{cmd} and Acc_{para} assess how closely the reconstructed CAD sequences resemble the original human-designed sequences. The final CAD models are also quantitatively evaluated against the ground-truth CAD models using Median Chamfer Distance (Med CD), which measures the geometric similarity between the reconstructed and ground-truth models. Additionally, we use the Invalid Rate (IR) to evaluate the percentage of CAD sequences output that fail to compile.

Baselines. DeepCAD [36] provides a point cloud-conditioned generation scheme; therefore, we replace the point cloud encoder with a DINO-V2 image encoder and adaptive layers [36] to allow image-conditioned generation. HNC-CAD [39] supports the conditional generation of partial command CAD sequences, we retrain the networks by feeding the DINO image features to its original conditional encoder. However, since HNC-CAD directly generates explicit *loop*, *profile* and *solids* instead of commands, we do not include comparisons for command accuracy (Acc_{cmd}) and parameter accuracy (Acc_{para}) with this model. Img2CAD [7] is capable of generating CAD commands from images. Since their code is not publicly available, we use the performance metrics reported directly in their paper. Besides CAD generation methods, we also compare our approach with recent image-to-3D methods, such as One-2-3-45 [16], Wonder3D [18], and TripoSR [29]. Wonder3D [18] also utilizes normal information for 3D reconstruction. To ensure a fair comparison, we fine-tune One-2-3-45 [16] and Wonder3D [18] using our rendered DeepCAD data, while TripoSR [29] is a commercial model without publicly accessible training codes. Since they directly generate 3D shapes, we only evaluate the Median Chamfer Distance (Med CD) against these methods. In subsequent sections, $\dagger$ denotes models fine-tuned on our CAD data, while $*$ indicates models for which we replaced the original condition encoder with an image encoder and then retrained.

4.2. Quantitative Results

We compare our model with existing approaches in Table 1. On DeepCAD [36] dataset, the results show that our CAD-Crafter achieves high CAD sequence accuracy and significantly reduces the failure rate in various scenarios on both multi-view and single-view tasks. Our method outperforms DeepCAD [36], HNC-CAD [39] and Img2CAD [7] in each

Methods	$Acc_{cmd} \uparrow$	$Acc_{para} \uparrow$	Med CD $\downarrow$	IR $\downarrow$
DeepCAD test set (synthetic)				
DeepCAD _s *	77.72	65.30	0.126	0.123
DeepCAD _m *	79.62	66.75	0.113	0.106
HNC-CAD _s *	-	-	0.214	0.114
HNC-CAD _m *	-	-	0.208	0.101
Img2CAD	80.57	68.77	0.160	0.288
TripoSR	-	-	0.136	-
One-2-3-45 $\dagger$	-	-	0.151	-
Wonder3D $\dagger$	-	-	0.133	-
CADCrafter _s	83.23	71.82	0.049	0.042
CADCrafter _m	84.62	73.31	0.026	0.036
RealCAD Dataset (real-world)				
DeepCAD _s *	56.59	41.32	0.264	0.527
DeepCAD _m *	54.11	37.27	0.295	0.567
HNC-CAD _s *	-	-	0.276	0.147
HNC-CAD _m *	-	-	0.305	0.167
TripoSR	-	-	0.128	-
One-2-3-45 $\dagger$	-	-	0.147	-
Wonder3D $\dagger$	-	-	0.125	-
CADCrafter _s	81.23	64.16	0.082	0.087
CADCrafter _m	83.18	66.89	0.062	0.067

Table 1. Performance comparisons on the synthetic DeepCAD dataset and real-world RealCAD dataset where s denotes single-view and m denotes multi-view settings.

criterion, especially the invalid rate, demonstrating the robustness of our method. On RealCAD dataset, the performance of DeepCAD [36] and HNC-CAD [39] significantly declined across all criteria, indicating their overfitting to synthetic data and inability to generalize to real-world scenarios. This underscores the significant domain gap between rendered synthetic data and real-world captured data. In contrast, our method, despite being trained solely on synthetic data, generalizes effectively to real-world data with only a slight performance drop, maintaining high accuracy and low invalid rates.

We also benchmarked our method against advanced large-scale image-to-3D generative models One-2-3-45 [16], Wonder3D [18], and TripoSR [29], which demonstrated consistent performance across both synthetic and real-world data due to their generalizability. However, our method consistently outperformed these models in terms of geometric accuracy in both scenarios.

4.3. Qualitative Results

We compare our method with existing methods qualitatively in Figure 5. Our method successfully recovered the CAD command sequences from the single image input on both synthetic and real-world scenarios, while DeepCAD [36] and HNC-CAD [39] failed to produce meaningful shapes in both cases. Wonder3D [18] and TripoSR [29] generate better results than One-2-3-45 [16]. However, the generated shapes often exhibit unsmooth surfaces and lack precision. Additionally, they consistently fail to accurately reproduce

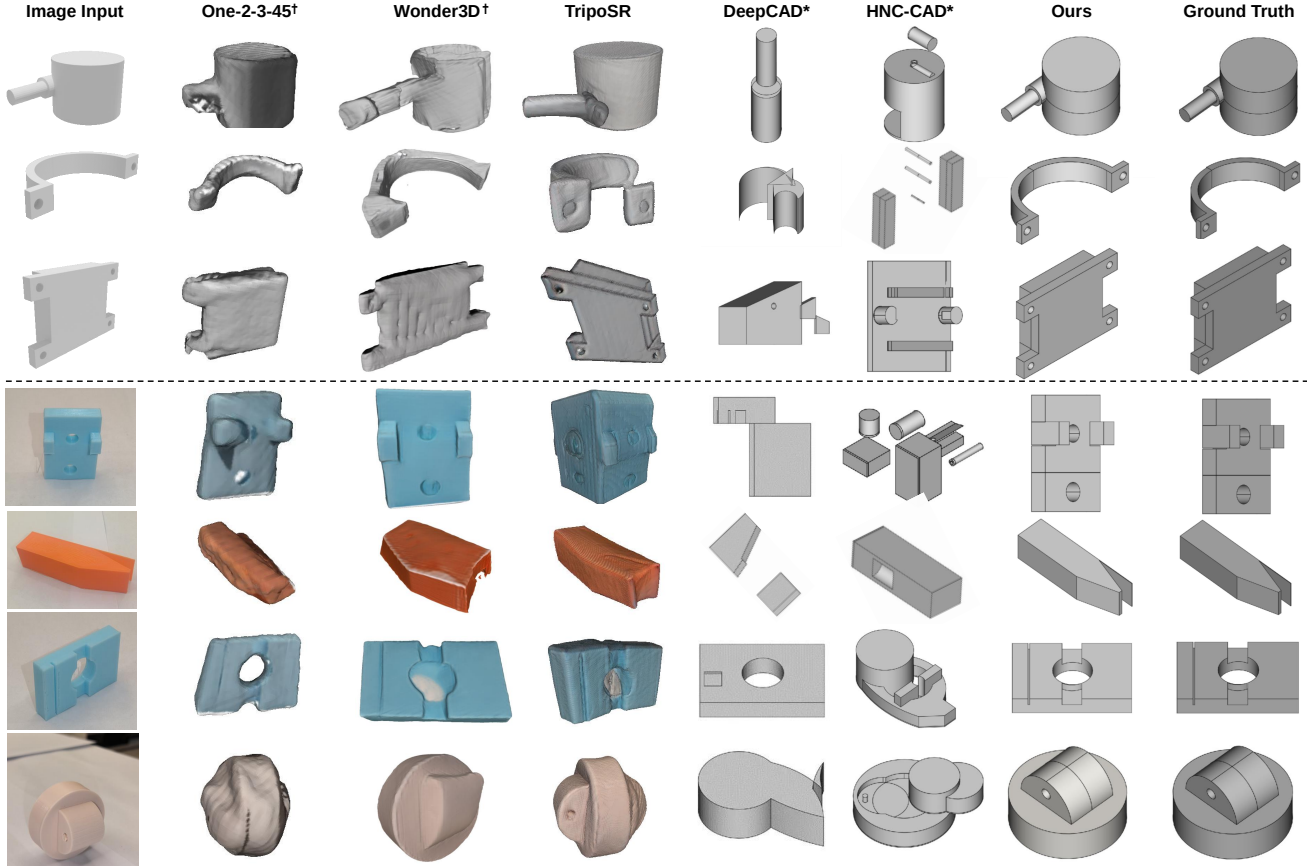


Figure 5. We compare the generated CAD models from single-view images with existing methods on two datasets: the upper part shows results on the DeepCAD renderings, and the lower part shows results on the real-world RealCAD dataset.

standard geometric shapes such as rectangles and circles, making these shape approximations unsuitable for manufacturing applications.

4.4. Ablation Studies of Different Modalities

To study the impact of different modalities, we sequentially train and test our model on different combinations of modal-

Inputs	ACC _{cmd} ↑	ACC _{para} ↑	Med CD ↓	IR ↓
<i>DeepCAD test set (synthetic)</i>				
RGB	83.26	72.09	0.029	0.037
Normal	83.14	72.63	0.032	0.039
Depth	83.06	72.28	0.036	0.041
RGB+Depth+Normal	85.18	74.86	0.023	0.031
Depth+Normal (Ours)	84.62	73.31	0.026	0.036
<i>RealCAD Dataset (real-world)</i>				
RGB	77.92	49.73	0.219	0.26
Normal	82.57	65.77	0.106	0.073
Depth	82.17	64.78	0.102	0.087
RGB+Depth+Normal	78.67	52.78	0.192	0.227
Depth+Normal (Ours)	83.18	66.89	0.062	0.067

Table 2. Ablation studies on various geometric modalities with multi-view inputs reveal that while RGB slightly enhances performance on synthetic data, it significantly reduces the model’s generalizability.

ities or each modality alone. As shown in Table 2, the results show that since CAD models inherently lack textures, using solely rendered images, normals, or depth maps yields similar outcomes on the synthetic DeepCAD dataset. Each modality captures unique information: normals emphasize the relationships between surfaces, while depth maps focus on object scale. Thus, combining all three modalities leads to the best performance while tested on synthetic setting.

However, testing the trained models on real-world images reveals that normals and depth maps, which focus solely on geometric characteristics, are not impacted by the domain gap introduced by the textures of the CAD model. While incorporating an RGB image as input can slightly improve performance on synthetic data, the significant difference between synthetic RGB data and real images markedly reduces the model’s generalizability when trained with RGB inputs. Therefore, to enhance the model’s generalizability, we exclude RGB images during both training and testing.

4.5. Ablations of Different Components

We perform ablation studies of different components in the single-view setting in Table 3.

Methods	ACC _{cmd} ↑	ACC _{para} ↑	Med CD ↓	IR ↓
CADCrafter _{w/o-L_{Geo}}	81.89	69.98	0.056	0.059
CADCrafter _{scratch}	80.61	68.62	0.079	0.078
CADCrafter _{w/o-L_{distill}}	81.12	69.83	0.068	0.072
CADCrafter _{w/o-L_{dpo}}	81.64	69.32	0.072	0.081
CADCrafter	83.23	71.82	0.049	0.042

Table 3. Ablation studies of different components on the DeepCAD dataset with single-view inputs.

Geometric Encoder. In CADCrafter_{w/o-L_{Geo}}, we replace our geometric encoder with concatenated DINO features, processed through a 3-layer MLP to produce the conditional embedding. This change led to a noticeable decrease in performance, demonstrating that our transformer-based geometric encoder effectively consolidates geometric information across different modalities.

From Scratch. In CADCrafter_{scratch}, we train the single-view geometry encoder and the diffusion model from scratch. The decline in performance indicates that our sequential training strategy, which leverages multi-view knowledge, benefits the single-view configuration.

Multi-view distillation. In CADCrafter_{w/o-L_{distill}}, we train the single-view geometry encoder alongside a diffusion model that was pre-trained in a multi-view setting but without employing our distillation loss. The performance decline confirms that the distillation loss effectively transfers comprehensive knowledge from the multi-view encoder to the single-view encoder.

Direct Preference Optimization. In CADCrafter_{w/o-L_{dpo}}, we remove the DPO fine-tuning alone. The results demonstrate that DPO effectively lowers the CAD code invalid rate (IR) and improves command accuracy. This underscores the effectiveness of our code checker in helping the model learn more accurate code patterns with geometric constraints.

4.6. Applications of CADCrafter

Creating digital twins with CAD models largely benefits the manufacturing industry and Embodied AI simulation. Currently, we train exclusively on synthetic datasets of mechanical parts but have successfully demonstrated the con-

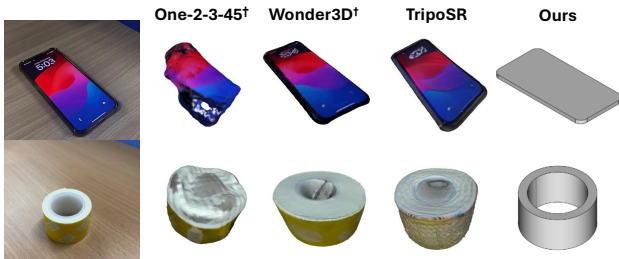


Figure 6. We compare our image-to-CAD results on unseen general objects with image-to-3d methods.

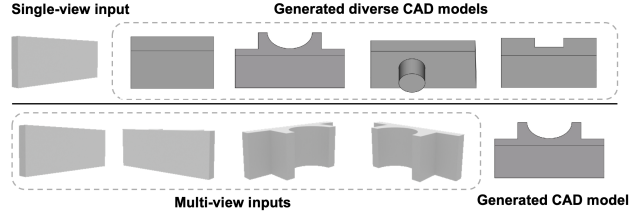


Figure 7. In the single-view setting, CADCrafter can generate diverse shapes for the unseen parts. Given the multi-view input, CADCrafter is able to reconstruct more accurate shapes.

version of casually captured real-world objects into editable CAD models, as illustrated in Figure 6. To our knowledge, we are the first to showcase this capability. While the current generation complexity is limited by existing datasets, advancements in the field should enable the conversion of more complex objects into precise CAD models.

Since the single-view images cannot capture the complete information of the objects, it is desirable to provide various choices given the partial observations. In the first row of Figure 7, we demonstrate that our model can offer various CAD models with different unseen parts in the single-view image. Users can further choose and edit these generated results to suit their specific needs. Additionally, as illustrated in the second row of Figure 7, multi-view images provide more sufficient information about the shape geometry, and our model can generate more specific models when precise results are needed.

5. Conclusions and Future Work

We introduce CADCrafter, a latent diffusion model that converts images into CAD command sequences using geometric information. Trained solely on synthetic data, CADCrafter generalizes effectively to real-world images and unseen object types. Our geometric encoder bridges synthetic-real domain gaps by capturing diverse shape information and distilling multi-view knowledge into a single-view encoder, enhancing single-view performance. Additionally, we propose an automated code checker using direct preference optimization to incorporate CAD compiler feedback, improving geometric accuracy. We also contribute a new dataset of unconstrained images of 3D-printed CAD models with corresponding commands for validation. Future work includes incorporating physical properties essential for manufacturing and extending the model with text-based editing capabilities.

Acknowledgements

This research work is supported by the Agency for Science, Technology and Research (A*STAR) under its MTC Programmatic Funds (Grant No. M23L7b0021).

References

- [1] Autocad. <https://www.autodesk.com/products/autocad>. 3
- [2] Fusion 360. <https://www.autodesk.com/products/fusion-360>.
- [3] Onshape. <http://onshape.com>. 3
- [4] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022. 2, 5
- [5] Cheng Chen, Zhenshan Tan, Qingrong Cheng, Xin Jiang, Qun Liu, Yudong Zhu, and Xiaodong Gu. Utc: A unified transformer with inter-task contrastive learning for visual dialog. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, pages 18103–18112, 2022. 3
- [6] Cheng Chen, Xiaofeng Yang, Fan Yang, Chengzeng Feng, Zhoujie Fu, Chuan-Sheng Foo, Guosheng Lin, and Fayao Liu. Sculpt3d: Multi-view consistent text-to-3d generation with sparse 3d prior. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10228–10237, 2024. 3
- [7] Tianrun Chen, Chunyan Yu, Yuanqi Hu, Jing Li, Tao Xu, Runlong Cao, Lanyun Zhu, Ying Zang, Yong Zhang, Zejian Li, et al. Img2cad: Conditioned 3d cad model generation from single image with structured visual geometry. *arXiv preprint arXiv:2410.03417*, 2024. 3, 6
- [8] Gene Chou, Yuval Bahat, and Felix Heide. Diffusion-sdf: Conditional generative modeling of signed distance functions. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2262–2272, 2023. 3
- [9] Yuan Dong, Qi Zuo, Xiaodong Gu, Weihao Yuan, Zhengyi Zhao, Zilong Dong, Liefeng Bo, and Qixing Huang. GPLD3D: latent diffusion of 3d shape generative models by enforcing geometric and physical priors. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, pages 56–66. IEEE, 2024. 2
- [10] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. Lrm: Large reconstruction model for single image to 3d. *arXiv preprint arXiv:2311.04400*, 2023. 1, 3
- [11] Mohammad Sadil Khan, Elona Dupont, Sk Aziz Ali, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-signet: Cad language inference from point clouds using layer-wise sketch instance guided attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4713–4722, 2024. 3
- [12] Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2cad: Generating sequential cad models from beginner-to-expert level text prompts. *arXiv preprint arXiv:2409.17106*, 2024. 3
- [13] Changjian Li, Hao Pan, Adrien Bousseau, and Niloy J Mitra. Sketch2cad: Sequential cad modeling by sketching in context. *ACM Transactions on Graphics (TOG)*, 39(6):1–14, 2020. 2
- [14] Pu Li, Jianwei Guo, Xiaopeng Zhang, and Dong-Ming Yan. Secad-net: Self-supervised cad reconstruction by learning sketch-extrude operations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16816–16826, 2023. 1, 2
- [15] Pu Li, Jianwei Guo, Huibin Li, Bedrich Benes, and Dong-Ming Yan. Sfmcad: Unsupervised cad reconstruction by learning sketch-based feature modeling operations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4671–4680, 2024. 1, 2
- [16] Minghua Liu, Chao Xu, Haian Jin, Linghao Chen, Mukund Varma T, Zexiang Xu, and Hao Su. One-2-3-45: Any single image to 3d mesh in 45 seconds without per-shape optimization. *Advances in Neural Information Processing Systems*, 36, 2024. 1, 6
- [17] Minghua Liu, Chong Zeng, Xinyue Wei, Ruoxi Shi, Linghao Chen, Chao Xu, Mengqi Zhang, Zhaoning Wang, Xiaoshuai Zhang, Isabella Liu, et al. Meshformer: High-quality mesh generation with 3d-guided reconstruction model. *arXiv preprint arXiv:2408.10198*, 2024. 3
- [18] Xiaoxiao Long, Yuan-Chen Guo, Cheng Lin, Yuan Liu, Zhiyang Dou, Lingjie Liu, Yuexin Ma, Song-Hai Zhang, Marc Habermann, Christian Theobalt, et al. Wonder3d: Single image to 3d using cross-domain diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9970–9980, 2024. 1, 6
- [19] Weijian Ma, Minyang Xu, Xueyang Li, and Xiangdong Zhou. Multicad: Contrastive representation learning for multi-modal 3d computer-aided design models. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 1766–1776, 2023. 3
- [20] Weijian Ma, Shuaiqi Chen, Yunzhong Lou, Xueyang Li, and Xiangdong Zhou. Draw step by step: Reconstructing cad construction sequences from point clouds via multimodal diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 27154–27163, 2024. 1, 2, 3, 6
- [21] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shangwen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023. 3
- [22] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022. 3
- [23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024. 2, 5

- [24] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 1 (2):3, 2022. 4
- [25] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. 3
- [26] Wenhao Shen, Wanqi Yin, Hao Wang, Chen Wei, Zhonggang Cai, Lei Yang, and Guosheng Lin. Hmr-adapter: A lightweight adapter with dual-path cross augmentation for expressive human mesh recovery. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 6093–6102, 2024. 3
- [27] Yawar Siddiqui, Antonio Alliegro, Alexey Artemov, Tatiana Tommasi, Daniele Sirigatti, Vladislav Rosov, Angela Dai, and Matthias Nießner. Meshgpt: Generating triangle meshes with decoder-only transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19615–19625, 2024. 3
- [28] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. 4
- [29] Dmitry Tochilkin, David Pankratz, Zexiang Liu, Zixuan Huang, Adam Letts, Yangguang Li, Ding Liang, Christian Laforte, Varun Jampani, and Yan-Pei Cao. Triposr: Fast 3d object reconstruction from a single image. *arXiv preprint arXiv:2403.02151*, 2024. 6
- [30] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017. 3, 4
- [31] Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lou, Senthil Purushwalkam, Stefano Ermon, Caiming Xiong, Shafiq Joty, and Nikhil Naik. Diffusion model alignment using direct preference optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8228–8238, 2024. 2, 5
- [32] Hanxiao Wang, Mingyang Zhao, Yiqun Wang, Weize Quan, and Dong-Ming Yan. Vq-cad: Computer-aided design model generation with vector quantized diffusion. *Computer Aided Geometric Design*, 111:102327, 2024. 2
- [33] Jiacheng Wei, Hao Wang, Jiashi Feng, Guosheng Lin, and Kim-Hui Yap. Taps3d: Text-guided 3d textured shape generation from pseudo supervision. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16805–16815, 2023. 3
- [34] Karl DD Willis, Pradeep Kumar Jayaraman, Joseph G Lambourne, Hang Chu, and Yewen Pu. Engineering sketch generation for computer-aided design. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2105–2114, 2021. 1, 2
- [35] Karl DD Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics (TOG)*, 40(4): 1–24, 2021. 3
- [36] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6772–6782, 2021. 1, 2, 3, 5, 6
- [37] Xianghao Xu, Wenzhe Peng, Chin-Yi Cheng, Karl DD Willis, and Daniel Ritchie. Inferring cad modeling sequences using zone graphs. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6062–6070, 2021. 1, 2
- [38] Xiang Xu, Karl DD Willis, Joseph G Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman, and Yasutaka Furukawa. Skexgen: Autoregressive generation of cad construction sequences with disentangled codebooks. *arXiv preprint arXiv:2207.04632*, 2022. 3
- [39] Xiang Xu, Pradeep Kumar Jayaraman, Joseph G Lambourne, Karl DD Willis, and Yasutaka Furukawa. Hierarchical neural coding for controllable cad model generation. *arXiv preprint arXiv:2307.00149*, 2023. 3, 6
- [40] Xiang Xu, Joseph G. Lambourne, Pradeep Kumar Jayaraman, Zhengqing Wang, Karl D. D. Willis, and Yasutaka Furukawa. Brep-gen: A b-rep generative diffusion model with structured latent geometry. *ACM Trans. Graph.*, 43(4): 119:1–119:14, 2024. 2
- [41] Guandao Yang, Xun Huang, Zekun Hao, Ming-Yu Liu, Serge Belongie, and Bharath Hariharan. Pointflow: 3d point cloud generation with continuous normalizing flows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4541–4550, 2019. 3
- [42] Hu Ye, Jun Zhang, Sibio Liu, Xiao Han, and Wei Yang. Ip-adapter: Text compatible image prompt adapter for text-to-image diffusion models. *arXiv preprint arXiv:2308.06721*, 2023. 2
- [43] Wei Yin, Chi Zhang, Hao Chen, Zhipeng Cai, Gang Yu, Kaixuan Wang, Xiaozhi Chen, and Chunhua Shen. Metric3d: Towards zero-shot metric 3d prediction from a single image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9043–9053, 2023. 3, 6